READ FIRST · FOR CLAUDE CODE (AGENT CODE AND AGENT GIT) AND AGRESH

Spareway — Plan, Architecture & Spec

What we build, in which order, by which agent, and how every part works — from the run order to the database. No ready-made code: Claude writes it from this.

PRODUCT	BUILD	VERSION
Simple load + route planner for small Indian truck owners, in English and Hindi	Phase 0 alone, then 4 phases with 2 agents side by side · 31 Oct – 1 Nov	Final v6 · 29 Oct 2026

0. Start here

0.1 The documents

File	What it is	Who reads it
Spareway_01_Plan_Architecture_and_Spec.pdf (this)	The plan: run order, phases, rules, architecture, sign-in, Hindi, database, every feature, setup	Both agents, every phase
Spareway_02_Design_System.pdf	Mandatory look: simple-UI rules, colours, components, every screen with its Haulix reference picture	Both agents, every phase
Spareway_03_Engineering_Reference.pdf	Exact names, shapes, numbers: contracts, database functions, errors, demo data + Hindi dictionary, tests, git, templates	Both agents, every phase
Spareway_04_Team_Responsibilities.pdf	What Bhavya, Yash and Vansh prepare and do	The team
docs/phases/Phase<n>_<Code|Git>_....pdf (9 files)	One phase part: questions, folders, build order, checks, hand-off	The agent running that part
supabase/spareway_database.sql	The tested database (tables, security, functions, demo data)	Agent Code (Phase 0)

If two documents disagree: behaviour and data → this PDF; look and wording → Design System; names and numbers → Engineering Reference; still unclear → ask Agresh.

0.2 Run order (exactly in this order)

#	When	Agent Code (desktop app, local)	Agent Git (browser, cloud)
1	Before Day 1	**Kick-off session**: paste the kick-off prompt (0.3). It reads everything and lists questions. No code.	—
2	Day 1 morning	**Phase 0** — setup and foundation, alone (≈ 3 h). Ends with tag `v0`.	— (waits)
3	Day 1 midday	**Phase 1·Code** Trucks, trips, cargo	**Phase 1·Git** Packing engine, 3D load plan, map style
4		**Merge 1** (fresh session) → tag `v1`	—
5	Day 1 evening	**Phase 2·Code** Loader link, route service, live map	**Phase 2·Git** Route planning and trip cost

#	When	Agent Code (desktop app, local)	Agent Git (browser, cloud)
6		**Merge 2** → tag `v2` (mentor checkpoint 1)	—
7	Day 2 morning	**Phase 3·Code** Camera + Hindi voice	**Phase 3·Git** Extra loads + marketplace
8		**Merge 3** → tag `v3` (mentor checkpoint 2)	—
9	Day 2 midday	**Phase 4·Code** Offline, public link, final check (includes the final merge → `v4-final`)	**Phase 4·Git** Earnings and analytics

Every tag is a complete product you can demo. Start each phase with a **new session** (`/clear`) and the prompt from that phase PDF. Start the Git part first (it runs on its own in the cloud), then the Code part.

0.3 Kick-off prompt (Agent Code, before Day 1)

```
Read all PDFs in docs/ (in parts if needed) and docs/phases/. Do not write code. Then: 1) summarise the plan in 10 lines, 2) list every question you need answered for Phase 0 and anything missing for later phases, 3) confirm you understand the rules in section 1. Wait for my answers. Tomorrow I will say "Start Phase 0".
```

0.4 Phases at a glance

Phase	Agent Code (local)	Agent Git (cloud)	You can show after it
Phase 0	Setup and foundation (runs alone) — F1, F2, F3, F4, F5, F6	— (Phase 0 runs alone)	Sign up with email or Google → "Setting up your demo trucks..." → Home with 6 trucks, 1 trip on the road, problems and activity → tap हिंदी and everything (labels and demo data) is in Hindi → Settings saves costs.
Phase 1	Trucks, trips and cargo — F7, F8, F9	Packing engine, 3D load plan, map style — F10, F11, F12	Add a truck from a preset, create a trip with 3 drops, add cargo (demo trip shows 104 units), open the Load plan step and see the truck packed in 3D, activate it. Any trip gets a 3D plan with reasons for anything that didn't fit; "Use this plan" activates it; the map style is ready for Phase 2.
Phase 2	Loader link, route service, live map — F13, F14, F15	Route planning and trip cost — F16	Send the loader link by QR/WhatsApp → a loader without login loads the truck step by step on a phone in Hindi; routes come from the route service; all trucks on the live map. Trip step Route: three routes with full cost, breaks and no-entry waits; choose one; ETAs fill in; driver gets a WhatsApp summary.
Phase 3	Add cargo by camera and Hindi voice — F17, F18	Extra loads on the way and marketplace — F19, F20	Owner or loader photographs boxes or says "दस बाई दस का बॉक्स, ऊँचाई बीस, पचास पीस, नाज़ुक है" → checks the items → they appear in the cargo list; re-plan. Extra loads step lists profitable loads on the way and rejects four with plain reasons; "Add to trip" adds stops and cargo and re-plans; owners can post public loads.
Phase 4	Offline, public link, final check — F21, F22	Earnings and analytics — F23	Final product: installs on the phone, survives network drops, runs on a public HTTPS link where any judge can sign up and get the demo trucks; everything checked in English and Hindi. Trip step Earnings shows "You earn ₹..." with the gain from extra loads; /earnings shows the fleet's week in big numbers and two simple charts.

0.5 The two agents

	Agent Code	Agent Git
Runs	Claude desktop app → Code → Local, folder `spareway` on Agresh's laptop · Account 1	claude.ai/code in the browser, repo `spareway`, environment `spareway-cloud` · Account 2
Can	Supabase (MCP), secrets, dev server, phone testing over the hotspot, Vercel, merging	npm install, code, tests, build, push its branch, read the docs, web search
Cannot	—	Reach Supabase, use secrets, deploy, see the app with real data, use camera or mic
So it builds	Screens that need the database, phone or keys; Edge Functions; merges and real checks	Logic with strong tests and screens built on typed `api.ts` + test data (fixture mode)
Proves its work	Clicks through the real app on laptop + phone	Tests + build + a **"Code must verify"** list that Agent Code runs at the merge
Usage limit	Account 1's 5-hour limit	Account 2's (cloud shares the limit of its own account — keep the two agents on different accounts)

0.6 What every phase contains

0·Code Setup and foundation (runs alone) — main — AGENT CODE · local app

Item	Detail
Builds	F1 Project setup, database, sample data · F2 Sign-in and accounts · F3 Design system, Hindi button and shared helpers · F4 App shell · F5 Home (dashboard) · F6 Settings
Writes only in	Everything (it creates the project): root configs, `src/app`, `src/lib`, `src/types`, `src/i18n`, `src/components`, `src/features/{home,auth,settings}`, `supabase/functions/_shared`, `public/brand`, `public/icons`, `CLAUDE.md`, `docs/PROGRESS.md`, `docs/DECISIONS.md`.
Never touches	—
Starts after · runs with · time	Repository with the docs committed · nothing — runs alone · ≈ 3 h · Opus (Sonnet if usage is low)
Already built when it starts	Only the docs, `supabase/spareway_database.sql`, Vansh's CSVs in `data/pois/` and Bhavya's logo in `public/brand/`.
You can show after it	Sign up with email or Google → "Setting up your demo trucks…" → Home with 6 trucks, 1 trip on the road, problems and activity → tap हिंदी and everything (labels and demo data) is in Hindi → Settings saves costs.

1·Code Trucks, trips and cargo — code/phase1-trucks-trips — AGENT CODE · local app

Item	Detail
Builds	F7 Trucks · F8 Trips and stops · F9 Cargo list
Writes only in	`src/features/{trucks,trips,cargo}`, `supabase/sql/phase1code_*.sql`.
Never touches	`src/features/load-planner`, `public/map` (Agent Git). Shared core files from Phase 0.
Starts after · runs with · time	Phase 0 (tag v0) · 1·Git — Packing engine, 3D load plan, map style · ≈ 3 h · Sonnet
Already built when it starts	Project, database (+ real POIs), contracts, module + slot system, router. Sign-in pages, accounts, demo trucks for every new company. Design system incl. Hindi button, `td()` demo-data translation, NextStepCard, StepTabs, BigTile, MapView. Shell (rail / bottom tabs, top bar with Hindi button, bell, avatar menu), Home with slots `home.actions` + `home.widgets`, Settings with slot `settings.sections`.
You can show after it	Add a truck from a preset, create a trip with 3 drops, add cargo (demo trip shows 104 units), open the Load plan step and see the truck packed in 3D, activate it.

1·Git Packing engine, 3D load plan, map style — git/phase1-load-plan — AGENT GIT · cloud

Item	Detail
Builds	F10 Packing engine · F11 3D load plan · F12 Dark map style
Writes only in	`src/features/load-planner`, `public/map`, `tools/map-preview.html`.

Item	Detail
Never touches	`src/features/{trucks,trips,cargo}` (Agent Code is building them). Read trucks, stops and cargo through your own `load-planner/api.ts` using the committed types.
Starts after · runs with · time	Phase 0 (tag v0) · 1·Code — Trucks, trips, cargo · ≈ 3 h · Opus for the engine, Sonnet for the rest
Already built when it starts	Project, database (+ real POIs), contracts, module + slot system, router. Sign-in pages, accounts, demo trucks for every new company. Design system incl. Hindi button, `td()` demo-data translation, NextStepCard, StepTabs, BigTile, MapView. Shell (rail / bottom tabs, top bar with Hindi button, bell, avatar menu), Home with slots `home.actions` + `home.widgets`, Settings with slot `settings.sections`.
You can show after it	(after the merge) Any trip gets a 3D plan with reasons for anything that didn't fit; “Use this plan” activates it; the map style is ready for Phase 2.

2·Code Loader link, route service, live map

code/phase2-loader-map · AGENT CODE · local app

Item	Detail
Builds	F13 Loader link and loader page (no login) · F14 Route service (Edge Function route-plan) · F15 Live map
Writes only in	`src/features/{loader,live-map}`, `supabase/functions/route-plan`, `supabase/sql/phase2code_*.sql`.
Never touches	`src/features/routes` (Agent Git).
Starts after · runs with · time	Merge 1 (tag v1) · 2·Git — Route planning and trip cost · ≈ 3 h · Sonnet
Already built when it starts	Project, database (+ real POIs), contracts, module + slot system, router. Sign-in pages, accounts, demo trucks for every new company. Design system incl. Hindi button, `td()` demo-data translation, NextStepCard, StepTabs, BigTile, MapView. Shell (rail / bottom tabs, top bar with Hindi button, bell, avatar menu), Home with slots `home.actions` + `home.widgets`, Settings with slot `settings.sections`. Trucks (/trucks, slot `truck.tabs`), Trips (/trips, trip page with StepTabs, slots `trip.steps` + `trip.actions`), Cargo list (slot `cargo.addMethods`). Packing engine + 3D load plan (public API pack, fitAdditional, TruckScene, TopView, SideView, usePlanForTrip); dark map style in `public/map/`.
You can show after it	Send the loader link by QR/WhatsApp → a loader without login loads the truck step by step on a phone in Hindi; routes come from the route service; all trucks on the live map.

2·Git Route planning and trip cost

git/phase2-routes · AGENT GIT · cloud

Item	Detail
Builds	F16 Route planning and trip cost
Writes only in	`src/features/routes`.
Never touches	`src/features/{loader,live-map}`, `supabase/functions/route-plan` (Agent Code). Call route-plan only through its contract; mock it in tests.
Starts after · runs with · time	Merge 1 (tag v1) · 2·Code — Loader link, route service, live map · ≈ 3 h · Sonnet

Item	Detail
Already built when it starts	Project, database (+ real POIs), contracts, module + slot system, router. Sign-in pages, accounts, demo trucks for every new company. Design system incl. Hindi button, `td()` demo-data translation, NextStepCard, StepTabs, BigTile, MapView. Shell (rail / bottom tabs, top bar with Hindi button, bell, avatar menu), Home with slots `home.actions` + `home.widgets`, Settings with slot `settings.sections`. Trucks (/trucks, slot `truck.tabs`), Trips (/trips, trip page with StepTabs, slots `trip.steps` + `trip.actions`), Cargo list (slot `cargo.addMethods`). Packing engine + 3D load plan (public API pack, fitAdditional, TruckScene, TopView, SideView, usePlanForTrip); dark map style in `public/map/`.
You can show after it	(after the merge) Trip step Route: three routes with full cost, breaks and no-entry waits; choose one; ETAs fill in; driver gets a WhatsApp summary.

3·Code Add cargo by camera and Hindi voice

`code/phase3-camera-voice` **AGENT CODE · local app**

Item	Detail
Builds	F17 Add cargo by camera · F18 Hindi voice input
Writes only in	`src/features/smart-input`, `supabase/functions/ai-cargo`, `supabase/sql/phase3code_*.sql`.
Never touches	`src/features/extra-loads` (Agent Git).
Starts after · runs with · time	Merge 2 (tag v2) · 3·Git — Extra loads and marketplace · ≈ 3 h · Sonnet
Already built when it starts	Project, database (+ real POIs), contracts, module + slot system, router. Sign-in pages, accounts, demo trucks for every new company. Design system incl. Hindi button, `td()` demo-data translation, NextStepCard, StepTabs, BigTile, MapView. Shell (rail / bottom tabs, top bar with Hindi button, bell, avatar menu), Home with slots `home.actions` + `home.widgets`, Settings with slot `settings.sections`. Trucks (/trucks, slot `truck.tabs`), Trips (/trips, trip page with StepTabs, slots `trip.steps` + `trip.actions`), Cargo list (slot `cargo.addMethods`). Packing engine + 3D load plan (public API pack, fitAdditional, TruckScene, TopView, SideView, usePlanForTrip); dark map style in `public/map/`. Loader step + /load/:token loader page (slot `loader.actions`); route-plan Edge Function deployed; live map /map. Route step with three costed routes and driver WhatsApp (public API costRoute, planSchedule, RouteLine, useSelectedRoute).
You can show after it	Owner or loader photographs boxes or says “दस बाई दस का बॉक्स, ऊँचाई बीस, पचास पीस, नाज़ुक है” → checks the items → they appear in the cargo list; re-plan.

3·Git Extra loads on the way and marketplace

`git/phase3-extra-loads` **AGENT GIT · cloud**

Item	Detail
Builds	F19 Extra loads on the way · F20 Loads marketplace
Writes only in	`src/features/extra-loads`.
Never touches	`src/features/smart-input`, `supabase/functions/ai-cargo` (Agent Code).

Item	Detail
Starts after · runs with · time	Merge 2 (tag v2) · 3·Code — Camera and Hindi voice · ≈ 3 h · Opus for the logic, Sonnet for the UI
Already built when it starts	Project, database (+ real POIs), contracts, module + slot system, router. Sign-in pages, accounts, demo trucks for every new company. Design system incl. Hindi button, `td()` demo-data translation, NextStepCard, StepTabs, BigTile, MapView. Shell (rail / bottom tabs, top bar with Hindi button, bell, avatar menu), Home with slots `home.actions` + `home.widgets`, Settings with slot `settings.sections`. Trucks (/trucks, slot `truck.tabs`), Trips (/trips, trip page with StepTabs, slots `trip.steps` + `trip.actions`), Cargo list (slot `cargo.addMethods`). Packing engine + 3D load plan (public API pack, fitAdditional, TruckScene, TopView, SideView, usePlanForTrip); dark map style in `public/map/`. Loader step + /load/:token loader page (slot `loader.actions`); route-plan Edge Function deployed; live map /map. Route step with three costed routes and driver WhatsApp (public API costRoute, planSchedule, RouteLine, useSelectedRoute).
You can show after it	(after the merge) Extra loads step lists profitable loads on the way and rejects four with plain reasons; “Add to trip” adds stops and cargo and re-plans; owners can post public loads.

4·Code Offline, public link, final check

`code/phase4-offline-deploy` **AGENT CODE · local app**

Item	Detail
Builds	F21 Offline and low-data mode · F22 Public link and final check
Writes only in	`src/features/offline`, the `// PWA` block in `vite.config.ts`, `vercel.json`, `scripts/`; offline-queue wiring inside your own `cargo` and `loader` code.
Never touches	`src/features/analytics` (Agent Git).
Starts after · runs with · time	Merge 3 (tag v3) · 4·Git — Earnings and analytics · ≈ 3 h · Sonnet
Already built when it starts	Project, database (+ real POIs), contracts, module + slot system, router. Sign-in pages, accounts, demo trucks for every new company. Design system incl. Hindi button, `td()` demo-data translation, NextStepCard, StepTabs, BigTile, MapView. Shell (rail / bottom tabs, top bar with Hindi button, bell, avatar menu), Home with slots `home.actions` + `home.widgets`, Settings with slot `settings.sections`. Trucks (/trucks, slot `truck.tabs`), Trips (/trips, trip page with StepTabs, slots `trip.steps` + `trip.actions`), Cargo list (slot `cargo.addMethods`). Packing engine + 3D load plan (public API pack, fitAdditional, TruckScene, TopView, SideView, usePlanForTrip); dark map style in `public/map/`. Loader step + /load/:token loader page (slot `loader.actions`); route-plan Edge Function deployed; live map /map. Route step with three costed routes and driver WhatsApp (public API costRoute, planSchedule, RouteLine, useSelectedRoute). Camera + Hindi voice cargo (owner + loader), ai-cargo Edge Function deployed. Extra loads step + /loads marketplace + Home widget “Extra earnings available”.
You can show after it	Final product: installs on the phone, survives network drops, runs on a public HTTPS link where any judge can sign up and get the demo trucks; everything checked in English and Hindi.

4·Git Earnings and analytics

git/phase4-earnings AGENT GIT · cloud

Item	Detail
Builds	F23 Earnings and analytics
Writes only in	`src/features/analytics`.
Never touches	`src/features/offline`, `vite.config.ts`, `vercel.json` (Agent Code).
Starts after · runs with · time	Merge 3 (tag v3) · 4·Code — Offline, public link, final check · ≈ 2 h · Sonnet
Already built when it starts	Project, database (+ real POIs), contracts, module + slot system, router. Sign-in pages, accounts, demo trucks for every new company. Design system incl. Hindi button, `td()` demo-data translation, NextStepCard, StepTabs, BigTile, MapView. Shell (rail / bottom tabs, top bar with Hindi button, bell, avatar menu), Home with slots `home.actions` + `home.widgets`, Settings with slot `settings.sections`. Trucks (/trucks, slot `truck.tabs`), Trips (/trips, trip page with StepTabs, slots `trip.steps` + `trip.actions`), Cargo list (slot `cargo.addMethods`). Packing engine + 3D load plan (public API pack, fitAdditional, TruckScene, TopView, SideView, usePlanForTrip); dark map style in `public/map/`. Loader step + /load/:token loader page (slot `loader.actions`); route-plan Edge Function deployed; live map /map. Route step with three costed routes and driver WhatsApp (public API costRoute, planSchedule, RouteLine, useSelectedRoute). Camera + Hindi voice cargo (owner + loader), ai-cargo Edge Function deployed. Extra loads step + /loads marketplace + Home widget “Extra earnings available”.
You can show after it	(after the merge) Trip step Earnings shows “You earn ₹…” with the gain from extra loads; /earnings shows the fleet's week in big numbers and two simple charts.

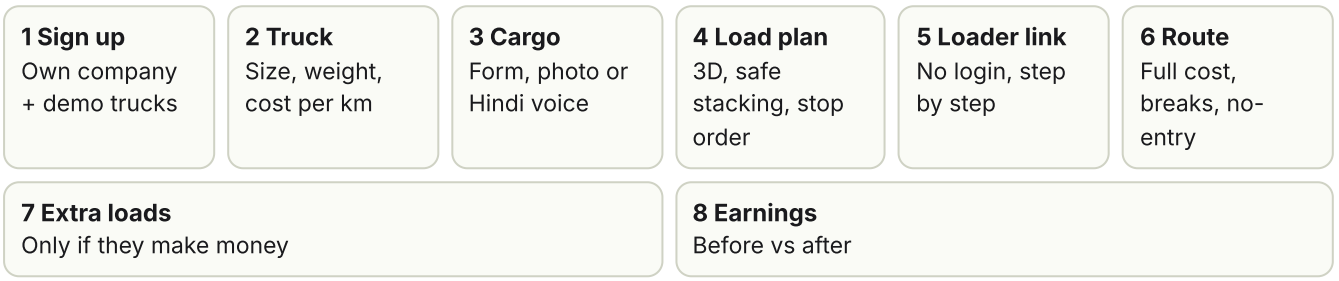
1. Rules for Claude Code (both agents, every session)

1. **Read first**: this PDF, the Design System, the Engineering Reference, your phase PDF, CLAUDE.md and docs/PROGRESS.md. Know who you are: local with Supabase MCP = Agent Code; cloud = Agent Git.
2. **Ask, then plan, then build**: ask the questions in your phase PDF (plus yours, max ~6, multiple choice), show a plan, wait for approval.
3. **Never guess**: database names only from the SQL file and `src/types/database.ts`; library APIs from the installed version in node_modules; external APIs (Supabase Auth, OpenRouteService, Gemini) checked in current docs and noted in DECISIONS.md.
4. **Stay in your lane**: write only in the folders your phase owns (0.6, 4.4). The two parts of a phase never share a folder. Shared files from Phase 0 (`src/app`, `src/lib`, `src/types/domain.ts`, `src/components/shell`, `src/index.css`, `package.json`, `vite.config.ts`, `CLAUDE.md`) are not edited later — need a change? write a CONTRACT CHANGE REQUEST (Engineering Reference 12.4). You may add new files to `src/components/common`.
5. **Plug in only through slots** and your feature's `module.tsx`; import other features only through their `index.ts`.
6. **Simple and bilingual**: follow the Design System's simple-UI rules; every string in English and Hindi; demo data through `td()`.
7. **Honest numbers**: no invented numbers in the UI; Sample / Simulated / AI / Estimated badges where due.
8. **Database**: only Agent Code touches Supabase. Changes are additive SQL files `supabase/sql/phase<n><code|git>_*.sql`; Agent Code applies them and regenerates types. Agent Git never connects and never hand-edits `database.ts`.
9. **Secrets** only as Supabase secrets, never in files, commits or logs. The browser only has the anon key.
10. **Finish properly**: `npm run check` + `npm run build` pass (also without .env.local); Verify list done (Git: "Code must verify" list written); PROGRESS.md + DECISIONS.md updated; small commits pushed to your branch; only Agent Code merges to `main`.
11. **Stuck twice** on the same error → stop, explain, ask. Long session → update PROGRESS.md, then `/compact`.

2. Product

Problem. Small truck owners in India (1–5 trucks) often drive with part of the truck empty: loading is not planned, extra cargo on the way is hard to find, and routes are chosen by habit. That wastes money and adds trips.

Spareway is a very simple phone-first web app, in English and Hindi, that helps them fill the truck safely and earn more per trip.



User	Access	Does
Owner (the only account type)	Email + password or Google; each signup = one company	Everything

User	Access	Does
Loader	No account — secret link (QR / WhatsApp), works 48 h	Loads step by step, reports problems, adds cargo by photo or voice
Driver	No account	Gets the route, breaks and stops as a WhatsApp message

> **Demo story (every new company gets it):** trip **TRP-2026-0031**, truck **MP09 HG 4821** (32 ft container, 15 t) from **Manesar** to **1 Jaipur → 2 Ahmedabad → 3 Mumbai (Bhiwandi)** with 104 units of home appliances (2,638 kg). Spareway packs it in 3D, sends the loader link, compares three NH48 routes, finds extra loads on the way (e.g. LD-1045 Vadodara → Bhiwandi) and rejects four bad ones with plain reasons. Earnings go up.

Simple for everyone: big buttons, plain words, a “Next step” card on every trip, one tap to Hindi. **Honest:** AI only suggests cargo, a person confirms; packing, money and routes are tested code; sample and simulated data are marked.

3. System architecture

Part	Where	Job
React web app (Vite, PWA in Phase 4)	Browser — phone and laptop	All screens; packing engine in a Web Worker; route cost and extra-load logic; speech; camera; Hindi
Supabase Auth	Supabase	Email + password, Google; sessions; emails through Brevo SMTP
Supabase Postgres + security rules	Supabase (Mumbai)	All data, one company per account; functions for multi-step writes and the loader link
Supabase Storage · Realtime	Supabase	Private cargo photos · live updates for owners
Edge Function route-plan (2·Code)	Supabase	OpenRouteService truck routes with cache
Edge Function ai-cargo (3·Code)	Supabase	Gemini reads photos / speech → cargo suggestions
Brevo · Google OAuth · OpenRouteService · Gemini · OpenFreeMap · Vercel	External (free)	Emails · Google sign-in · routes · AI · map tiles · public hosting

Flow	Steps
Sign up	Form → Supabase Auth → trigger creates company + profile + settings → email (Brevo) → /auth/callback → app calls `seed_sample_data()` once → Home
Show data	Page → React Query hook → feature `api.ts` → supabase-js → security rules (own company) → screen; demo values through `td()` in Hindi
Load plan	Cargo → Web Worker `pack()` → 3D → save `load_plans` → `activate_load_plan` → loading steps
Loader link	`create_loader_link` → QR / WhatsApp `/load/<token>` → loader page polls `loader_get_job`, writes `loader_update_step` / `loader_add_cargo` → owner sees it live
Routes	Route step → route-plan → costs + breaks with POIs → `route_options` → choose → stop ETAs
Extra loads	Open listings + route line → fit, cost, profit → `accept_extra_load` → new plan
Photo / voice	Capture → ai-cargo (or rule parser for voice) → check items → owner insert or `loader_add_cargo`

4. Code architecture

4.1 Layers

Layer	Lives in	Rule
Pages	`features/*/pages`	One per route, lazy-loaded, no direct Supabase calls
Components	`features/*/components`, `components/common`, `components/ui`	Typed props, strings via `t()`, demo values via `td()`
Hooks	`features/*/hooks`	React Query + realtime
API	`features/*/api.ts`	Only place that calls supabase-js; throws AppError
Logic	`features/*/lib`, `load-planner/engine`	Pure TypeScript with tests
Backend	`supabase/`	SQL + Edge Functions

4.2 Packages (Phase 0 installs all; nobody adds more)

Area	Packages
Build	vite, @vitejs/plugin-react, typescript, @vitejs/plugin-basic-ssl
UI	tailwindcss, @tailwindcss/vite, shadcn/ui, lucide-react, clsx, tailwind-merge, class-variance-authority, motion, sonner
App	react-router, @supabase/supabase-js, @tanstack/react-query, react-hook-form, zod, @hookform/resolvers, i18next, react-i18next, date-fns, date-fns-tz
Fonts	@fontsource-variable/inter, Noto Sans Devanagari from Fontsource (check the exact name with `npm view`)
Charts · 3D · maps	recharts · three, @react-three/fiber, @react-three/drei · maplibre-gl, react-map-gl, @turf/turf
Utilities	comlink, browser-image-compression, qrcode.react
Offline (used in Phase 4)	vite-plugin-pwa, @tanstack/react-query-persist-client, @tanstack/query-sync-storage-persister
Dev	vitest, @testing-library/react, @testing-library/jest-dom, jsdom, prettier

4.3 Folders

```
spareway/
├─ CLAUDE.md  .env.local (laptop only, never committed)  .env.example  vercel.json (Phase 4)
├─ docs/ 4 PDFs · phases/ 9 PDFs · PROGRESS.md · DECISIONS.md
├─ data/pois/*.csv                route data (Vansh)
├─ supabase/ spareway_database.sql · sql/phase<n><code|git>_*.sql · functions/{_shared,route-plan,ai-
cargo}
├─ public/ brand/ (logo) · icons/ (favicon, PWA) · map/ (dark + lite style, map icons)
└─ src/
   ├─ main.tsx  index.css
   ├─ app/        App, router, providers, modules, slots, auth/
   ├─ i18n/       setup · common/{en,hi}.json · data/hi.json (demo data in Hindi)
   ├─ lib/        supabase, env, queryKeys, format, units, geo, lowData, realtime, errors
   ├─ types/      database.ts (generated) · domain.ts (contracts)
   ├─ components/ ui/ · common/ · shell/
   └─ features/   home · auth · settings · trucks · trips · cargo · load-planner · loader · live-map
.
                  routes · smart-input · extra-loads · offline · analytics
```

4.4 Who owns which folder

Phase part	Agent	Owns
0·Code	Code	Everything (it creates the project): root configs, `src/app`, `src/lib`, `src/types`, `src/i18n`, `src/components`, `src/features/{home,auth,settings}`, `supabase/functions/_shared`, `public/brand`, `public/icons`, `CLAUDE.md`, `docs/PROGRESS.md`, `docs/DECISIONS.md`.
1·Code	Code	`src/features/{trucks,trips,cargo}`, `supabase/sql/phase1code_*.sql`.
1·Git	Git	`src/features/load-planner`, `public/map`, `tools/map-preview.html`.
2·Code	Code	`src/features/{loader,live-map}`, `supabase/functions/route-plan`, `supabase/sql/phase2code_*.sql`.
2·Git	Git	`src/features/routes`.
3·Code	Code	`src/features/smart-input`, `supabase/functions/ai-cargo`, `supabase/sql/phase3code_*.sql`.
3·Git	Git	`src/features/extra-loads`.
4·Code	Code	`src/features/offline`, the `// PWA` block in `vite.config.ts`, `vercel.json`, `scripts/`; offline-queue wiring inside your own `cargo` and `loader` code.
4·Git	Git	`src/features/analytics`.

4.5 Feature modules and slots

```
src/features/<name>/
  module.tsx   FeatureModule: routes, publicRoutes, nav, slot entries
  index.ts     public API for other features
  pages/ components/ hooks/ api.ts schemas.ts lib/ (+ __tests__) types.ts
  i18n/en.json i18n/hi.json   (namespace = <name>)
```

The app finds modules with `import.meta.glob('../features/*/module.tsx', { eager: true })`. **Slots** let a feature add a button, tab or card to a page it doesn't own. An empty slot shows nothing, so every phase looks complete.

Slot	Host page (owner)	Props	Filled by
home.actions	Home (0·Code)	—	1·Code New trip · 1·Git Plan load · 2·Code Live map · 3·Git Find extra loads
home.widgets	Home (0·Code)	—	3·Git Extra earnings available · 4·Git This week
settings.sections	Settings (0·Code)	—	4·Code Save map for offline
truck.tabs	Truck page (1·Code)	truckId	1·Git Load layout · 2·Code Location
trip.steps	Trip page (1·Code)	tripId	1·Git Load plan (2) · 2·Code Loader (3) · 2·Git Route (4) · 3·Git Extra loads (5) · 4·Git Earnings (6)
trip.actions	Trip page (1·Code)	tripId	(small extra buttons; none planned)
cargo.addMethods	Cargo list (1·Code)	tripId, onAdded	3·Code Scan with camera · Speak
loader.actions	Loader page (2·Code)	token, job	3·Code Scan with camera · Speak

4.6 Conventions

Topic	Rule
Names	Components PascalCase, hooks useThing, DB snake_case, i18n keys `namespace:section.key`, branches as in each phase PDF
Server state	React Query only; keys in `lib/queryKeys.ts`; staleTime 30 s; sign-out clears the cache
UI state	Local state; app-wide contexts only for auth, language, low-data
Realtime	`useRealtime(table, filter)` only invalidates queries; owners only (loader page polls)
Forms	react-hook-form + zod; inline errors; toast on success
Errors	AppError {code, message}; plain messages from Engineering Reference 5; error screen per route
Units	Stored cm, kg, km, ₹; shown ft for trucks, cm for cargo, tonnes over 10,000 kg, ₹1,24,500, IST
Language	Every string in en + hi; demo data via `td()`; alerts/activity from kind + meta templates
Fixture mode (Git)	`?fixture=seed` in dev builds feeds Engineering Reference 7 data instead of api.ts

4.7 Routes

Path	Phase	Screen	Access
/login · /signup · /forgot-password · /reset-password · /auth/callback	0·Code	Sign-in pages	public
/load/:token	2·Code	Loader page (phone, no login)	token
/	0·Code	Home	owner
/settings	0·Code	Settings	owner
/trucks · /trucks/:truckId	1·Code	Trucks · Truck page	owner

Path	Phase	Screen	Access
/trips · /trips/new · /trips/:tripId?step=…	1·Code	Trips · New trip · Trip page with steps	owner
/trips/:tripId/plan	1·Git	3D load plan full screen	owner
/map	2·Code	Live map	owner
/loads · /loads/new	3·Git	Loads marketplace · Post a load	owner
/earnings	4·Git	Earnings and analytics	owner

5. Sign-in and accounts (Phase 0)

One account type: **Owner**. Each signup creates its own **company**; every row has `company_id`, so companies never see each other's data. Loaders never log in.

Method	How	Notes
Email + password	`signUp` / `signInWithPassword`	Password ≥ 8 with a letter and a number. Confirmation email via Brevo; if email fails on the day, turn Confirm email off
Google	`signInWithOAuth({ provider: 'google', options: { redirectTo: origin + '/auth/callback' } })`	Same button on Log in and Sign up; company named "<Name> Transport" (editable)

Screen	Fields	Behaviour
Log in /login	Logo, Hindi button, Continue with Google, or, email, password, Log in, links Forgot password · Create account	Go to `?next` or Home. Wrong password → "Email or password is wrong"; not confirmed → message + Resend email; too many tries → "Please wait a minute"
Sign up /signup	Google, or, your name, company name, email, password, confirm, Create account	`options.data = { full_name, company_name }`, `emailRedirectTo = origin + '/auth/callback'`. Session back → straight in; else "Check your email" (Resend after 60 s)
Forgot password	Email, Send link	Always the same message ("If an account exists, we sent a link")
Set new password	New password, confirm, Save	From the email link; expired → message + link to Forgot password
Auth callback	"Signing you in…"	Let supabase-js finish (PKCE code exchange if `code` in URL — check installed docs); show `error_description`; go to next or Home

- **useAuth()** = `{ session, user, profile, company, settings, status: 'loading'|'signedOut'|'preparing'|'ready'|'error', signInWithPassword, signUp, signInWithGoogle, sendPasswordReset, updatePassword, resendConfirmation, signOut, refreshProfile }`.
- **Guards**: RequireAuth around the shell (→ /login?next=…); PublicOnly around login/signup/forgot (→ Home); reset, callback and /load/:token always reachable.
- **First login**: if `company.sample_seeded_at` is empty → "Setting up your demo trucks…" → `seed_sample_data()` → Home. Missing profile → error + Retry.
- **Sign out** clears cached data. Session expired → "Please log in again".

5.1 Loader link (Phase 2·Code)

Step	What happens
Owner sends	Trip step Loader → `create_loader_link(trip_id)` → 36-character token, valid 48 h → QR, Copy, WhatsApp ("Spareway loading job TRP-2026-0031: <link> — valid till 2 Nov, 6:00 pm"), Turn off (`disable_loader_link`). A new link replaces the old one
Works while	Link on AND under 48 h old AND trip planned or loading. Re-plans keep the same link (the page always shows the current plan)
Loader page	`loader_get_job` every 5 s while visible; writes only through `loader_update_step` and `loader_add_cargo`; LINK_INVALID → "This link has ended"

Step	What happens
Security	Loaders can't read tables; every function checks the token; problems and added cargo create owner alerts

5.2 Security checklist

- Security rules on every table (own company); marketplace shows public listings to all.
- Only the anon key in the browser; service role only inside Edge Functions.
- route-plan needs a user session; ai-cargo needs a session or a valid loader token.
- Internal SQL helpers locked (tested).
- Password reset never reveals if an email exists; nothing secret in logs.

6. Hindi and simple language

- **One-tap Hindi button** (“हिंदी” / “English”) in the top bar, on every sign-in page and on the loader page. One tap switches the whole app instantly; saved in the profile (owner) or on the device (loader). `<html lang>` follows.
- **Everything switches**: labels, buttons, messages, errors, empty states, WhatsApp texts, and the **demo data** — place names, cargo names, truck models, client names, driver names, sample problems and activity (dictionary in Engineering Reference 7.6, file `src/i18n/data/hi.json`, helper `td()`).
- **Problems and activity** are shown from `kind` + `meta` with templates in both languages (e.g. loader_issue → “TRP-2026-0031 · स्टेप 12: डिब्बा फटा”); stored text is only a fallback.
- **What stays as typed**: anything a user types (own truck numbers, own place names, notes). Numbers use Latin digits in both languages; truck numbers and trip codes never change.
- **Plain words**: “Space used” not “volume utilisation”, “Balance OK” not “centre of gravity”, “Profit” not “net benefit”; the Design System glossary (section 10) is mandatory.
- **Voice**: Hindi, Hinglish and English sentences understood (F18).

7. Database (`supabase/spareway_database.sql`, tested)

Part 1 = 18 tables + security rules + storage + realtime. Part 2 = views + functions. Part 3 = global sample route points. Tested on PostgreSQL 16 with Supabase-like roles (two companies isolated, loader link without login, internal functions locked, Hindi template values stored).

Table	Key columns
companies · profiles · company_settings	name, city, phone, sample_seeded_at · full_name, email, preferred_lang · diesel, driver wage, value of time, handling, waiting, delay penalty, minimum profit (min_margin_inr), corridor_km, road_factor, rest rules, avg speed, checkpoint delay, support_ratio
trucks	reg_no, model, body_type, internal L/W/H cm, payload_kg, axles, mileage, cost_per_km_inr, status (active/idle/maintenance/offline), driver name/phone, last position, fuel, is_sample
trips · trip_stops	code, truck, status (planned/loading/in_transit/completed/cancelled), departure, base_freight_inr, selected_route_id, loader_token + expiry + enabled · seq (0 = origin), type, name, city, lat/lng, delivery window, eta, status
cargo_items	label, client, L/W/H cm, weight per unit, quantity, pickup/drop stop, fragile, upright_only, stackable, max_top_load, priority, source (manual/camera/voice/extra_load/seed), added_by_loader, image_path, confidence
load_plans · loading_progress	version, status, engine_version, input_hash, placements, unplaced, metrics · step_index, placement_key, status, note
route_options · route_cache · pois	variant, km, minutes, geometry, costs, halts, warnings, is_selected · cache (server only) · tolls/halts/checkpoints/city entry (global)
freight_listings · trip_extra_loads	visibility public/company, code, shipper, goods, pickup/drop, size, weight, qty, flags, offer ₹, dates, status, accepted_by_company · detour, costs, profit, minimum viable rate, feasibility
alerts · activity_log	kind, severity, title, message, **meta** (template values for Hindi), resolved · kind, message, entity, **meta**
fleet_daily_stats · ai_scans	per day: space used %, km, on-time %, revenue, cost, extra-load revenue · AI call log (via_loader)

Function	Caller	Does
handle_new_user (trigger) · current_company_id()	Supabase	New user → company + profile + settings · caller's company
seed_sample_data() · reset_sample_data()	0·Code	Demo data once per company · reset demo, keep own trips
next_trip_code() · next_listing_code()	1·Code · 3·Git	TRP-2026-0032… · LD-2001…
activate_load_plan(plan_id)	1·Git	Plan active, loading steps, activity (meta version + space %)
create_loader_link · disable_loader_link	2·Code	48 h link · turn off
loader_get_job · loader_update_step · loader_add_cargo	2·Code, 3·Code (no login)	Read job · step done/problem (owner alert with meta) · add cargo (owner alert with meta)
loader_trip_for_token	ai-cargo (server only)	Checks a loader token
accept_extra_load(id, pickup_after_seq, drop_after_seq)	3·Git	Adds/reuses stops (2 km), adds cargo, marks listing taken, activity with meta

Function	Caller	Does
trip_overview · dashboard_stats · trip_financials (views)	many	Read models, own company only

Full function signatures and errors: Engineering Reference section 3.

8. Features F1–F23 — full plan of each

F1 Project setup, database, sample data

Phase 0 · Code (alone) | AGENT CODE · local app

Goal. A clean project everyone builds on: scaffold, all packages, the database with security rules, the demo data every new company gets, and the shared contracts.

SCREENS

- No screen of its own. Leaves `npm run dev` showing the app shell.

DATA

Applies `supabase/spareway_database.sql` as 3 migrations (schema, functions, global_seed); writes `supabase/sql/phase0_pois_real.sql` from Vansh's CSVs in `data/pois/` and applies it; generates `src/types/database.ts` (committed — the cloud agent relies on it).

HOW IT WORKS

- Every package in Architecture 4.2, latest stable, lockfile committed; nobody adds packages later.
- `npm run build` must pass without `.env.local` (cloud build) — env is read lazily at runtime.
- Contracts exactly as Engineering Reference section 1: `domain.ts`, module + slot system, router with publicRoutes, query keys, AppError.
- POI CSV → SQL: validate India bounds and numeric rates; re-runnable; report counts in PROGRESS.md.
- Brand: Bhavya's logo → `public/brand/`; favicon + PWA icons (16, 32, 180, 192, 512, maskable 512) in `public/icons/`.

FILES

Root configs, `package.json`, `src/main.tsx`, `src/app/*`, `src/lib/*`, `src/types/*`, `supabase/functions/_shared/*`, `public/brand`, `public/icons`, `CLAUDE.md`, `docs/PROGRESS.md`, `docs/DECISIONS.md`.

EDGE CASES

- Supabase MCP fails twice → print SQL for the SQL Editor and continue.
- CSVs missing → keep the global sample POIs.

TESTS

- Module discovery + slot ordering; build without env; POI validator.

F2 Sign-in and accounts

Phase 0 · Code (alone) | AGENT CODE · local app

Goal. Anyone can sign up with email + password or Google; each signup becomes its own company with a ready demo fleet. Log in, forgot and reset password all work.

SCREENS

- /login, /signup (+ “Check your email” state), /forgot-password, /reset-password, /auth/callback — one centred card each, logo, Hindi button, big fields, one lime button (Architecture section 5).
- First login: full-screen “Setting up your demo trucks…” then Home.

DATA

Supabase Auth; trigger `handle_new_user` creates company + profile + settings; RPC `seed_sample_data()` once.

HOW IT WORKS

- `AuthProvider` + `useAuth()` with the exact API in Architecture 5.3; guards `RequireAuth` / `PublicOnly`; `?next=` kept.
- Friendly error codes (Engineering Reference 5) shown in plain English/Hindi.
- Redirect URLs from `window.location.origin` so localhost, LAN and Vercel all work.

FILES

`src/app/auth/*` (provider, guards, bootstrap), `src/features/auth/*` (pages).

EDGE CASES

- Email confirmation off → signup goes straight in.
- Google cancelled → back to login with a message.
- Seed fails → error screen with Retry (no loop).

TESTS

- Error mapping; guards; signup states (mocked).

F3 Design system, Hindi button and shared helpers

Phase 0 · Code (alone) | AGENT CODE · local app

Goal. Every screen looks the same, stays simple, and switches fully to Hindi with one tap.

SCREENS

- All components of Design System section 5 (incl. **LanguageButton**, **NextStepCard**, **StepTabs**, **BigTile**, **MapView** with TruckMarker + StopPin).
- **Hindi button**: one pill "हिंदी" / "English" in the top bar, on auth pages and on the loader page. One tap switches every label, message and the demo data instantly.

DATA

`profiles.preferred_lang` saves the choice; Hindi demo-data dictionary `src/i18n/data/hi.json` built from Engineering Reference 7.6.

HOW IT WORKS

- i18next with namespaces per feature + `common` + `data`; key-parity test en/hi.
- `td(text)` = translate a stored demo value (place, cargo, truck model, alert text) if the dictionary has it; user-typed values stay as typed.
- Alerts and activity are shown from `kind` + `meta` templates (both languages), stored text only as fallback.
- `lib/format` (₹ en-IN, km, kg/t, ft, IST, relative time in en/hi), `lib/units` (cm⇄ft/in), `lib/geo`. Numbers stay in Latin digits in both languages.
- `MapView`: MapLibre wrapper with style URL from env (until Phase 1·Git's dark style exists, use the OpenFreeMap style URL directly), fit-bounds, markers, controls.

FILES

`src/index.css`, `src/components/{ui,common}/*`, `src/i18n/*`, `src/lib/{format,units,geo,i18n,lowData}.ts`.

EDGE CASES

- Hindi text 30% longer → wraps, never overflows at 375 px.
- Unknown demo string → shown as stored.

TESTS

- 975 cm → "32.0 ft"; 124500 → "₹1,24,500"; td() on sample values; parity test.

F4 App shell

Phase 0 · Code (alone) AGENT CODE · local app

Goal. One simple frame for every owner page.

SCREENS

- Laptop: floating icon rail with labels on hover (Home, Trucks, Trips, Map, Loads, Earnings) + Settings at the bottom; top bar: page title, search (trucks by number, trips by code), Hindi button, bell with problem count, avatar menu (Low-data mode, Settings, Log out).
- Phone: bottom tabs Home · Trucks · Trips · Map · More; top bar with title, Hindi button, bell.
- 404 page; About dialog (what Spareway is, “Sample data is marked”).

DATA

Open alerts count; search on trucks/trips.

HOW IT WORKS

- Nav items come from feature modules; items of features not built yet don't appear.

FILES

`src/components/shell/*`, `src/app/{App,router,modules,slots}.tsx`.

EDGE CASES

- Missing env → full-page message naming the variable.
- Realtime down → refetch every 30 s.

TESTS

- Nav built from modules; hidden when absent.

F5 Home (dashboard)

Phase 0 · Code (alone) AGENT CODE · local app

Goal. The owner sees in five seconds what is happening and what to do next.

SCREENS

- Greeting + date. Three **BigTiles**: Trucks working (3 of 6), Trips on the road (1), Problems (count, red if any).
- **Quick buttons** (slot `home.actions`): New trip, Plan load, Live map, Find extra loads — appear as features arrive.
- **Problems** list (alerts, simple cards: icon, one-line title, truck/trip chip, “Done” button) and **Recent activity** (last 6, one line each).
- **This week** card: one simple bar chart “Truck space used per day” (Sample badge). Then slot `home.widgets`.

DATA

`dashboard_stats`, `alerts`, `activity_log` (realtime), `fleet_daily_stats`.

HOW IT WORKS

- Problems sorted critical → warning → info; “Done” resolves optimistically.

FILES

`src/features/home/*`.

EDGE CASES

- No problems → “All good today” in green.
- No stats → chart empty state.

TESTS

- Sorting; resolve invalidation.

F6 Settings

Phase 0 · Code (alone) AGENT CODE · local app

Goal. Account, company, the cost numbers every calculation uses, and resetting the demo.

SCREENS

- /settings as simple cards: My account (name, email, language, change password) · Company (name, city, phone) · Costs (diesel ₹/litre, driver ₹/hour, minimum profit ₹, how far off-route to look km — each with one-line help) · **More settings** (collapsed: handling, waiting, delay penalty, rest rules, stacking support) · Demo data (Reset) · slot `settings.sections`.

DATA

`profiles`, `companies`, `company_settings`; RPC `reset_sample_data()`.

HOW IT WORKS

- zod ranges: diesel 50–200, wages 0–2,000, corridor 5–100 km, support 0.5–1, break 0.5–12 h; sticky Save bar when changed.

FILES

`src/features/settings/*`.

EDGE CASES

- Google user → no password card.
- Reset asks to confirm: “Your own trips stay.”

TESTS

- Schema limits.

F7 Trucks

Phase 1 · Code AGENT CODE · local app

Goal. Save each truck once with its real inside size, payload and cost per km.

SCREENS

- /trucks: filter chips (All, Working, Free, Repair, Off) with counts; Board view on laptop (columns by status, Haulix reference 7) and list on phone; truck card: number + status, “32 ft · 15 t”, driver name + call button, current trip chip.
- Add / Edit truck (sheet, full screen on phone): number, model, **size preset** (14–24 ft, 32 ft SXL/MXL “typical — check your truck”), inside length/width/height in ft (cm switch), payload t, axles, mileage, cost per km (help: fuel, tyres, maintenance, driver), driver name/phone, status; small box preview.
- /trucks/:truckId: big number + status, model, buttons Repair / Off / Edit, lime “New trip for this truck”; tabs Overview · Trips · Problems · then slot `truck.tabs`.

DATA

`trucks`, `trip_overview`, `alerts`; status changes → `activity_log`.

HOW IT WORKS

- Saved in cm; shown in ft; presets are suggestions.

FILES

`src/features/trucks/*` (public API `index.ts`: useTruck, TruckChip).

EDGE CASES

- Duplicate number → field error.
- Truck with trips can't be deleted → suggest Off.

TESTS

- ft⇄cm round trip; schema.

F8 Trips and stops

Phase 1 · Code | AGENT CODE · local app

Goal. Create a trip with ordered stops, then follow it step by step.

SCREENS

- /trips: chips Planned · Loading · On the road · Done; cards “Manesar → Mumbai (Bhiwandi) · 3 drops”, truck chip, departure, space used.
- /trips/new: 3 simple steps — 1 Truck · 2 Stops (pick from ~60 hubs or the map; drag to reorder; optional delivery time) · 3 Departure + freight ₹ → Create.
- /trips/:tripId: header (code, status, truck, route line), big status button for the next step (Start loading / Leave / Mark delivered); **StepTabs** 1 Cargo · then slot `trip.steps` (Load plan, Loader, Route, Extra loads, Earnings) with ✓ when done; **NextStepCard** on top (“Next: plan the load”). Slot `trip.actions`.

DATA

`trips`, `trip_stops`, `activity_log`; RPC `next_trip_code()`.

HOW IT WORKS

- `trips/lib/places.ts`: hubs incl. every sample place at the sample coordinates.
- `unloadOrderForStops(stops)` exported.
- Leave (depart) ends the loader link (confirm).

FILES

`src/features/trips/*` (public API: places, StopChip, useTrip, useTripStops, unloadOrderForStops).

EDGE CASES

- Stops fail to save → remove the new trip, show error.
- Editing stops after planning → warn the plan must be redone.

TESTS

- unloadOrderForStops; wizard schema.

F9 Cargo list

Phase 1 · Code | AGENT CODE · local app

Goal. Record what goes on the truck, grouped by where it is dropped.

SCREENS

- Trip step 1 “Cargo”: two bars “Weight 18% of truck” and “Space 34% of truck” (booked, before packing); groups by drop stop (StopChip + count + kg); rows as simple cards on phone: name, size, kg × qty, icons for Fragile / Keep upright / Don't stack, “Added by loader” and AI badges.
- Add item (sheet): name, client, size (cm or inch), kg each, quantity, drop stop, pickup stop (if not origin), three yes/no switches with one-line help, priority. Edit, copy, delete.
- Buttons: lime “Add item”, then slot `cargo.addMethods` (camera, voice from Phase 3·Code).

DATA

`cargo_items` (realtime so loader-added items appear live); one `overload` alert when over payload.

HOW IT WORKS

- Totals: units Σqty; kg Σ(weight×qty); m^3 Σ(L×W×H×qty)/10^6.

FILES

`src/features/cargo/*` (public API: useCargoItems, CargoList).

EDGE CASES

- Too big for the truck → “Won't fit this truck” chip.

TESTS

- Totals (104 units / 2,638 kg on the demo trip).

F10 Packing engine

Phase 1 · Git | AGENT GIT · cloud

Goal. A safe loading plan, fast and identical every time.

SCREENS

- No screen. Pure TypeScript in a Web Worker.

DATA

Input from `buildPackItems(cargoRows, stops, truck)`; output saved to `load_plans` (Engineering Reference 1.4).

HOW IT WORKS

- Coordinates: x from cab (0) to doors, y across, z up (cm).
- Sort: unload order high → low, load order low → high, volume ↓, weight ↓, key ↑. Extreme-point placement (Crainic, Perboli & Tadei 2008); 6 orientations (2 if keep-upright).
- Valid only if inside, no overlap, under payload, fully supported by stackable non-fragile boxes at least as heavy (support ratio from settings), and stop order respected (nothing blocks a box that leaves earlier).
- Reasons when something doesn't fit: TOO_LARGE, OVER_PAYLOAD, NO_SPACE, NO_SUPPORT, ORDER_CONFLICT. Balance warnings (left/right, front/back), heavy-high warning.
- inputHash (SHA-256) → same input reuses the plan; `fitAdditional` keeps already-loaded boxes fixed.

FILES

`src/features/load-planner/engine/*` + fixtures from Engineering Reference 7.

EDGE CASES

- 0 items → empty plan.
- Float drift → epsilon 0.5 cm.

TESTS

- 8 cubes fill 100%; invariants on every result; determinism; fitAdditional; demo trip 104 placed, 0 unplaced, < 1.5 s.

F11 3D load plan

Phase 1 · Git | AGENT GIT · cloud

Goal. Show the plan in 3D like Haulix reference 4, explain what didn't fit, and make it the active plan.

SCREENS

- Trip step “Load plan” (slot `trip.steps`) + full screen /trips/:tripId/plan: big “Space used 66%” and “Weight 18%”, three plain tiles (Free floor 11 ft · Weight left 12.4 t · Balance OK), 3D truck with open doors, boxes coloured by stop, “Cab side” and “Door side” labels, buttons 3D / From top / From side, “Play loading order”.
- Right panel: Packages by stop (step ranges, expandable card with client, drop, weight, fragile note, “Show in truck”) and “Didn't fit” list with plain reasons and advice.
- Lime “Use this plan” (activate). Live loader progress later: loaded boxes solid, waiting faded, problem red.
- Slots: `trip.steps` Load plan, `home.actions` Plan load, `truck.tabs` Load layout.

DATA

Reads trucks, trips, stops, cargo, settings.support_ratio, load_plans, loading_progress (realtime) via its own `api.ts`; RPC `activate_load_plan`.

HOW IT WORKS

- Re-plan remaining after a loader problem: fitAdditional with loaded boxes fixed.
- Dev fixture mode `?fixture=seed` (dev builds only) renders without a database.

FILES

`src/features/load-planner/*` (public API: pack, fitAdditional, buildPackItems, TruckScene, TopView, SideView, usePlanForTrip).

EDGE CASES

- No cargo → "Add cargo first".
- No WebGL / low-data → top and side drawings only.

TESTS

- Hooks with mocked worker and api; fixture render.

F12 Dark map style

Phase 1 · Git | AGENT GIT · cloud

Goal. A calm dark map that matches the app, plus a lighter low-data version.

SCREENS

- `public/map/spareway-dark.json` and `spareway-lite.json` from an OpenFreeMap base style (colours: Design System 8); map icons (truck, stop pins, toll, dhaba, fuel, checkpoint, no-entry) in `public/map/icons/`; `tools/map-preview.html`.

DATA

None.

HOW IT WORKS

- Attribution always visible; labels readable at night brightness.

FILES

`public/map/*`, `tools/map-preview.html`.

EDGE CASES

- Base style URL changes → start from liberty/positron.

TESTS

- Style JSON loads in a MapLibre unit test.

F13 Loader link and loader page (no login)

Phase 2 · Code | AGENT CODE · local app

Goal. The owner sends a 48-hour secret link; a loader with no account loads the truck step by step on his phone.

SCREENS

- Trip step "Loader" (slot `trip.steps`): big QR code, Copy link, lime "Send on WhatsApp", "Valid till Fri 6:00 pm", Turn off / New link, and loading progress "38 of 104 loaded" with problems listed.
- /load/:token (no shell, Hindi first, Hindi button): truck + trip on top, progress bar, one big step card (item name, drop stop colour, size + kg, where to put it in words — "Cab side · Left · Floor" — and a small top/side drawing with the box highlighted), huge buttons "Done ✓", "Problem", "Back"; problem sheet (Missing · Damaged · Different size · Other + note); finish screen "Truck loaded ✓"; slot `loader.actions` (camera/voice from Phase 3·Code).
- Expired link page: "This link has ended. Ask the owner for a new one."

DATA

RPCs `create_loader_link`, `disable_loader_link`, `loader_get_job` (poll every 5 s while visible), `loader_update_step`.

HOW IT WORKS

- Position words from coordinates: x thirds (cab side / middle / door side), y (left / centre / right), z layers (floor / 2nd / 3rd).
- PLAN_CHANGED → reload and continue at the same step. Problems create owner alerts automatically.

FILES

`src/features/loader/*` (uses `TopView` / `SideView` from load-planner).

EDGE CASES

- No active plan yet → "Owner is still planning" and keep polling.
- Two loaders on one link → last tap wins.

TESTS

- positionWords; polling pause/resume.

F14 Route service (Edge Function route-plan)

Phase 2 · Code | AGENT CODE · local app

Goal. Truck routes from OpenRouteService through a secure, cached server function.

SCREENS

- No screen.

DATA

Supabase secret `ORS_API_KEY`; table `route_cache`; contract in Architecture 9.1.

HOW IT WORKS

- JWT required; variants fastest / shortest / no_tolls sent one after another (ORS limits 40/min, 2,000/day); cache by SHA-256 of the request; retry without truck restrictions if none found; verify ORS field names in current docs.

FILES

`supabase/functions/route-plan/*`.

EDGE CASES

- Quota / ORS down → ORS_QUOTA / ORS_ERROR; cached result if any.

TESTS

- Deployed with MCP; curl example from Engineering Reference 6.1 returns 3 routes for the demo trip; no session → 401.

F15 Live map

Phase 2 · Code | AGENT CODE · local app

Goal. See all trucks on one map, like Haulix reference 6 — simple numbers, no gauges.

SCREENS

- /map: filter chips with counts, toggles Show routes / Show problems; truck markers (lime when selected).
- Selected-truck card (top-left on laptop, bottom sheet on phone): number + status, route "Ahmedabad → Surat", progress bar "62%", "Arrives 2:20 pm · 118 km left", plain line "Speed 52 km/h · Fuel 54%", amber "Break due in 40 min" banner, call driver button.
- Map controls (zoom, re-centre, full screen); footer "Simulated — updates every 3 s". Slots `home.actions` Live map, `truck.tabs` Location.

DATA

`trucks` (realtime), in-transit trips with selected routes.

HOW IT WORKS

- Simulation (Simulated badge): trucks move along their route at the trip's speed ×60; speed and fuel derived.

FILES

`src/features/live-map/*`.

EDGE CASES

- Truck without a position → listed below the map.
- Tiles fail → markers on plain dark background + note.

TESTS

- Position along a line at time t.

F16 Route planning and trip cost

Phase 2 · Git | AGENT GIT · cloud

Goal. Compare three truck routes on full trip cost, plan breaks and city no-entry waits, and send the driver a WhatsApp summary.

SCREENS

- Trip step “Route” (slot `trip.steps`): departure time; map (lime = chosen route, grey = others, numbered stops, toll/dhaba icons); three simple route cards: name + “Best” badge, km, time, total ₹ big, one line “Diesel ₹31,200 · Tolls ₹9,800 · Driver ₹7,400”, “2 breaks · 14 tolls”, warnings in plain words (“Reaches Mumbai at 6:40 pm — no entry until 9 pm, wait 2 h 20 min”), lime “Choose”.
- Journey list (start, stops, breaks, waits with times) and driver message (Hindi/English) with “Send on WhatsApp”.

DATA

Calls `route-plan` (contract 9.1, mocked in tests); reads pois, settings, truck, stops via its own `api.ts`; writes `route_options`, `trips.selected_route_id`, stop ETAs, activity, `city_entry` alert (with meta).

HOW IT WORKS

- Tolls within 0.5 km × axle rate; checkpoints within 1 km × delay.
- planSchedule: break every break_after_drive_h at a dhaba/parking within 3 km ahead, overnight after max_drive_h_per_day, wait out no-entry windows.
- costRoute: diesel = km ÷ mileage × price; tolls; driver = hours × wage; waiting; Best = lowest total + time value.

FILES

`src/features/routes/*` (public API: costRoute, planSchedule, RouteLine, useSelectedRoute).

EDGE CASES

- Route service down → cached routes or a clear message; rest of the trip page works.

TESTS

- costing, schedule, tolls on a fixture line through the demo stops (no network).

F17 Add cargo by camera

Phase 3 · Code | AGENT CODE · local app

Goal. Photograph boxes; AI suggests items and sizes; a person checks and adds them — owner and loader.

SCREENS

- "Scan with camera" in `cargo.addMethods` and `loader.actions`: 1–3 photos with a tip ("Keep an A4 sheet next to the boxes"), Analyse, then the **Check items** sheet (editable rows, confidence chip, drop stop + kg required, "Add 3 items").

DATA

Edge Function `ai-cargo` (built here, contract 9.2, secrets GEMINI_API_KEY + GEMINI_MODEL); images stored in `cargo-images/<company>/<trip>/`; `ai_scans`; owner inserts `cargo_items`, loader calls `loader_add_cargo(token, items, 'camera')`.

HOW IT WORKS

- Compress to ≤ 1280 px, ≤ 700 KB each; validator: sizes 5–400 cm, counts across photos, never silently change values.

FILES

`supabase/functions/ai-cargo/*`, `src/features/smart-input/*`.

EDGE CASES

- AI slow/fails → retry or type it.
- Camera blocked → help text + type instead.

TESTS

- Validator; deployed function answers the example in Engineering Reference 6.2.

F18 Hindi voice input

Phase 3 · Code | AGENT CODE · local app

Goal. Add cargo by speaking Hindi, Hinglish or English — e.g. "दस बाई दस का बॉक्स, ऊँचाई बीस, पचास पीस, नाज़ुक है".

SCREENS

- "Speak" button next to camera (owner and loader): big hold-to-talk mic, live text, हिंदी/English switch; missing value → asks "ऊँचाई कितनी है?"; then the same Check items sheet; "Type instead" always available.

DATA

Rule parser first; `ai-cargo` voice mode only when unsure; owner insert / loader RPC with 'voice'.

HOW IT WORKS

- webkitSpeechRecognition hi-IN / en-IN; parser handles Devanagari digits, number words (एक...सौ, डेढ़, ढाई, सवा, साढ़े), बाय/by/x/×, cm/सेमी/इंच/फुट, नाज़ुक/सीधा रखें/ऊपर कुछ न रखें; meaning of "ऊपर से / नीचे से" from the user's answer.

FILES

`src/features/smart-input/voice/*`.

EDGE CASES

- Not Chrome/Edge → typed input.
- Nothing heard → "Didn't catch that — try again or type it".

TESTS

- ≥ 25 sentences (Vansh's list).

F19 Extra loads on the way

Phase 3 · Git | AGENT GIT · cloud

Goal. Find loads that fit the space left on the way, keep only the ones that make money, and add them in one tap.

SCREENS

- Trip step “Extra loads” (slot `trip.steps`): three tiles (Free floor · Weight left · Possible extra earning), a simple route strip (km line with stops and arcs for each load), load cards sorted by profit: goods + from → to, size/weight, offer ₹, cost ₹, **Profit ₹** big in green, one plain reason (“Take it: ₹10,450 extra after ₹3,550 costs — 18 km detour, 2 tolls”), “See in truck” (3D preview) and lime “Add to trip”.
- Collapsed “Not suitable (4)” with plain reasons (too far off route · goes the other way · too heavy · pays too little — needs ≥ ₹X).

DATA

Open listings (own demo + everyone's public), trip, stops, truck, active plan, settings, selected route; writes `trip_extra_loads`; RPC `accept_extra_load`; new plan via `fitAdditional` → activate.

HOW IT WORKS

- Corridor (corridor_km) + direction (drop after pickup along the route) + time + weight + space (fitAdditional).
- Detour ≈ 2 × (pickup + drop offset) × road factor + 3 km per new stop (exact for the top 3 via route-plan).
- Cost = detour km × cost/km + extra tolls + handling + waiting + delay penalties; profit = offer − cost; recommended if ≥ minimum profit, else “needs ≥ ₹X”.

FILES

`src/features/extra-loads/*`.

EDGE CASES

- Taken by another company meanwhile → “Just taken” and refresh.
- No route chosen yet → approximate line through stops + Approximate badge.

TESTS

- Fixture NH48 line: LD-1049 off route, LD-1050 wrong way, LD-1051 too heavy, LD-1052 too cheap, ≥ 3 of LD-1041…1048 recommended.

F20 Loads marketplace

Phase 3 · Git | AGENT GIT · cloud

Goal. Browse loads and post a public load other companies can take.

SCREENS

- /loads (nav “Loads”): chips Open · Taken · All; cards “Kishangarh → Ahmedabad · Marble tiles · 1.4 t · ₹16,000 · ready today”, badges Demo / Public, “Find a trip for this load”.
- /loads/new: simple form (who, what, from, to, size, weight, quantity, offer ₹, dates) → “Post for everyone” (code from `next_listing_code()`). Home slots: Find extra loads button + “Extra earnings available” widget.

DATA

`freight_listings`.

HOW IT WORKS

- Own open posts can be removed.

FILES

`src/features/extra-loads/pages/Loads*`.

EDGE CASES

- No loads → “No loads right now”.

TESTS

- Post form schema.

F21 Offline and low-data mode

Phase 4 · Code | AGENT CODE · local app

Goal. The app installs on a phone, keeps working through network drops and stays light on slow data.

SCREENS

- Install button in the avatar menu; offline banner “No internet — showing saved data, changes will be sent later”; update toast; Settings slot “Save map for offline (NH48)”.

DATA

React Query persistence; queued owner cargo adds and loader Done/Problem taps.

HOW IT WORKS

- vite-plugin-pwa: never cache auth, Edge Functions or loader job data.
- Low-data: lite map, 3D off by default, no animations, loader polling 10 s.

FILES

`src/features/offline/*`, `// PWA` block in `vite.config.ts`.

EDGE CASES

- Sign-out clears the saved data.

TESTS

- Queue replays in order.

F22 Public link and final check

Phase 4 · Code | AGENT CODE · local app

Goal. Spareway runs on localhost, on the venue network for phones and on a free public HTTPS link — and every feature is checked end to end.

SCREENS

- None new.

DATA

Vercel project from the GitHub repo; Supabase URL Configuration + Google origins updated (Architecture 10).

HOW IT WORKS

- `vercel.json` rewrites all paths to index.html; env vars = the three VITE_ values.
- Final QA: every phase Verify list and every “Code must verify” list on laptop + phone, English + Hindi, owner + loader.

FILES

`vercel.json`, `scripts/*`, runbook in PROGRESS.md.

EDGE CASES

- Email slow on the day → Confirm email off.

TESTS

- Fresh signup on the public URL gets the demo trucks.

F23 Earnings and analytics

Phase 4 · Git | AGENT GIT · cloud

Goal. Show the money story simply: what a trip earns, before and after extra loads, and how the fleet is doing.

SCREENS

- Trip step “Earnings” (slot `trip.steps`): one big number “You earn ₹32,450” with “+₹10,450 from extra loads”; three tiles “Space used 34% → 52%”, “Empty floor saved 6 ft”, “Cost per tonne-km ₹2.41 → ₹1.73”; simple list freight − costs = earning; “Share on WhatsApp”.
- /earnings (nav “Earnings”): period chips (7 / 30 days); three big numbers; two simple bar charts (money earned per day, space used per day); top trips list. Home widget “This week”.

DATA

`trip_financials`, `fleet_daily_stats`, `load_plans` via its own api.ts.

HOW IT WORKS

- net = freight + accepted offers − route cost − extra costs.

FILES

`src/features/analytics/*`.

EDGE CASES

- No route chosen → “Choose a route to see costs”.

TESTS

- Earnings fixture (Engineering Reference 7.7).

9. Edge Functions (built and deployed by Agent Code)

Both: CORS, zod input check, reply `{ ok: true, data }` or `{ ok: false, error: { code, message } }`, never log secrets. Deploy with the Supabase MCP (fallback `npx supabase functions deploy`). Agent Git calls them only through this contract and mocks them in tests.

9.1 route-plan (Phase 2·Code · secret ORS_API_KEY · owners only)

Field	Meaning
Auth	Valid user session (Authorization header), else 401
Request	`{ tripId, coordinates: [[lng,lat],…] (≤ 50), variants: ['fastest','shortest','no_tolls'], truck: { lengthM, widthM, heightM, weightT, axleloadT } }`
Work	ORS Directions v2, profile `driving-hgv`, GeoJSON; no_tolls = avoid tollways; one variant after another; cache by SHA-256 in `route_cache`; retry without restrictions if no route
Response	`{ routes: [{ variant, distanceKm, durationMin, geometry, restricted, cached }] }`; errors ORS_QUOTA, ORS_ERROR

9.2 ai-cargo (Phase 3·Code · secrets GEMINI_API_KEY, GEMINI_MODEL · owner or loader)

Field	Meaning
Auth	User session (owner, must own tripId) OR `loaderToken` checked with `loader_trip_for_token`
Request	`{ mode: 'image', tripId?, loaderToken?, images: [{ base64, mime }] (1–3, ≤ 700 KB each), reference: { name: 'A4 sheet', lengthCm: 29.7, widthCm: 21 }, lang }` or `{ mode: 'voice', …, transcript, lang }`
Work	Store photos at `cargo-images/<company>/<trip>/<uuid>.jpg`; Gemini with a JSON schema (check current field names); log in `ai_scans`
Response	`{ scanId, model, imagePaths, items: [{ label, lengthCm, widthCm, heightCm, quantity, fragile, uprightOnly, stackable, confidence, notes, countsPerView }], warnings }`; errors AUTH_REQUIRED, LINK_INVALID, TOO_LARGE, AI_TIMEOUT, AI_PARSE

10. Environment and external setup

Item	Value / where
`.env.local` (laptop only) and Vercel env	VITE_SUPABASE_URL · VITE_SUPABASE_ANON_KEY · VITE_MAP_STYLE_URL=/map/spareway-dark.json
Supabase secrets	ORS_API_KEY (Phase 2·Code) · GEMINI_API_KEY, GEMINI_MODEL (Phase 3·Code)
Supabase MCP (Agent Code only)	`claude mcp add --transport http supabase "https://mcp.supabase.com/mcp?project_ref=<ref>"` in the repo folder (local scope, not committed) → `/mcp` → Authenticate
Cloud environment `spareway-cloud` (Agent Git)	Network Custom = default list + `tiles.openfreemap.org`; setup script `[ -f package.json ] && npm ci || true`; no env vars, no secrets. Claude GitHub App installed on the repo

Supabase dashboard (Agresh, click by click)	Setting
Authentication → Providers → Email	On; Confirm email ON when Brevo works (OFF as fallback); min password 8
Authentication → Providers → Google	On; Client ID + Secret from Google Cloud (OAuth client Web, consent screen published, redirect URI `https://<ref>.supabase.co/auth/v1/callback`)
Authentication → URL Configuration	Site URL = Vercel URL (until Phase 4: http://localhost:5173). Redirect URLs: `http://localhost:5173/**`, `https://localhost:5173/**`, `https://<laptop-LAN-IP>:5173/**` and `:4173/**`, `https://<app>.vercel.app/**`
Authentication → Emails → SMTP	smtp-relay.brevo.com, 587, user = Brevo login, password = Brevo SMTP key, sender = verified Brevo sender, name Spareway
Authentication → Emails → Templates	“Confirm your Spareway account” / “Reset your Spareway password”, one English + one Hindi line
Authentication → Rate limits	Emails per hour ~100 on demo day

11. Screens and journeys

Screen	Route	Phase
Sign-in pages (5)	/login …	0·Code
Setting up demo trucks	full screen	0·Code
Home	/	0·Code
Settings	/settings	0·Code
Trucks · truck page · truck form	/trucks	1·Code
Trips · new trip · trip page (steps)	/trips	1·Code
Cargo list · add item	trip step 1	1·Code
3D load plan	trip step 2 · /trips/:id/plan	1·Git
Loader step · loader page	trip step 3 · /load/:token	2·Code
Live map	/map	2·Code
Route step	trip step 4	2·Git
Camera · voice · check items	cargo list + loader page	3·Code
Extra loads step	trip step 5	3·Git
Loads marketplace · post a load	/loads	3·Git
Earnings step · Earnings page	trip step 6 · /earnings	4·Git
Offline banner · install · update	everywhere	4·Code

Journey	Steps (must work end to end)
Owner first time	Sign up (or Google) → demo trucks → Home → tap हिंदी → open TRP-2026-0031 → Cargo → Load plan → Use this plan → Loader (QR) → Route → choose Best → Extra loads → add LD-1045 → Earnings up → add an item by Hindi voice → re-plan

Journey	Steps (must work end to end)
Loader (no account)	Scan QR / open WhatsApp link → Hindi steps → Done … → Problem (different size) → owner re-plans → page continues → Speak an extra box → Truck loaded ✓ → link ends after 48 h, departure or turn off
Returning owner	Log in → own data only → forgot password → reset → Settings → change diesel price → new costs

12. Not in this project (do NOT build)

- More account types, team members or invites; loader or driver accounts.
- Landing page, help pages, terms, a “try demo” button.
- Command palette, CSV import.
- Phone OTP login, account deletion, payments, real GPS tracking, push notifications, driver app.
- Languages other than English and Hindi; translating text users type.